Dynamic reliability block diagrams: Overview of a methodology

Article · January 2007

CITATIONS
24

READS
2,784

2 authors:

Salvatore Distefano
University of Messina
275 PUBLICATIONS **4,995** CITATIONS
SEE PROFILE

Antonio Puliafito
University of Messina
562 PUBLICATIONS **10,788** CITATIONS
SEE PROFILE

Risk, Reliability and Societal Safety – Aven & Vinnem (eds)
© 2007 Taylor & Francis Group, London, ISBN 978-0-415-44786-7

Dynamic reliability block diagrams: Overview of a methodology

Salvatore Distefano & Antonio Puliafito
University of Messina, Department of Mathematics, Engineering Faculty, Messina, Italy

ABSTRACT: Dependability evaluation is an important, often indispensable, step in design and analyze (critical) systems, acquiring importance with the systems complexity growth. When the complexity of a system is high and/or increases, for example automizing or expanding some parts, dynamic effects, not present or manifested before, could arise or become significant in terms of reliability/availability. The system could be affected by common cause failures, the system components could interfere each other or could become inter/sequence-dependent, effects due to load sharing arise and therefore should be considered, and so on. Moreover could be interesting to evaluate redundancy and maintenance policies. In those cases it is not possible to recur to notations as reliability block diagrams (RBD), fault trees (FT) or reliability graphs (RG) to represent the system, since the statistical independence assumption is not satisfied. Also more enhanced formalisms as dynamic FT (DFT) could not result adequate to the objective. To overcome those problems we developed a new formalism derived from RBD: the dynamic RBD (DRBD). In this paper we explain how to use the DRBD notation in system modeling and analysis, coming inside a methodology that, starting from the system structure, drives to the overall system availability evaluation following modeling and analysis phases. To do this we use an example drawn from literature, consisting of a multiprocessor distributed computing system. By this we also compare our approach with the DFT one.

1 INTRODUCTION

A system is a collection of components, subsystems and/or assemblies arranged according to a specific design in order to achieve acceptable performance and reliability levels. The types of components, their quantities, their qualities and the manner in which they are arranged within the system have a direct effect on the system reliability. The main objective of system reliability evaluation is the construction of a model (life distribution) that represents the times-to-failure of the entire system based on the life distributions of the components, subassemblies and/or assemblies (*"black boxes"*) from which it is composed (Reliasoft 2003).

Several are the approaches available to represent and analyze system reliability. One is the *simulation* (Monte Carlo (Gedam & Beaudet 2000), discrete event (Kamal & Ayyub 1999)), as alternatives there are many *analytic* approaches (Markov models and derived (Trivedi 2001, Veeraraghavan & Trivedi 1994, Bouissou & Bonc 2003), Petri nets (Malhotra & Trivedi 1995, Bobbio et al. 2003), stochastic reward nets (Zuppala et al. 1994, Malhotra & Trivedi 1995), Bayesian networks (Zhou et al. 2006, Montani et al. 2006) and so on). All of them are powerful reliability analysis methods, but, on the other hand, they are not much *"user friendly"*, in the sense that often it is really hard to obtain a model directly from the specifics, especially in case of complex systems: the modeler needs to represent components and reliability relationships in terms of states and transitions (Markov model), places, arcs and transitions (Petri nets), . . ., points of view not so close to the modeler as a specific notation could be. This fact therefore motivated the definition of ad-hoc reliability/availability modeling formalisms as *reliability block diagrams* (RBD) (Rausand & Høyland 2003), *fault trees* (FT) (Vesely et al. 1981) and *reliability graphs* (RG) (Sahner et al. 1996). Although RBD, RG and FT provide a view of the system close to the modeler, more readable and understandable than any other formalism, they are defined under an (heavy) assumption: the components must be *statistically independent*. They do not provide any elements or capabilities to model reliability interactions among components or subsystems, or to represent system reliability configuration alterations, aspects conventionally identified as *dynamic*. From the reliability point of view, it could be possible that, generally, a subsystem has some influence to other subsystems, characterizing the system reliability *dynamics*. Common examples of such relationships are: *load-sharing*, *standby redundancy*, *interferences*,

dependencies, *probabilistic and common mode failures*, ... Also the configuration of a system, considering the reliability aspects, could vary: a failed component/subsystem could be repaired (maintenance, *reliability growth model*), the system could be *phased mission*, and so on.

These argumentations awakened the dependability community to the need of new formalisms as the *dynamic fault trees* (DFT) (Boyd 1991, Dugan & Doyle 1996). DFT extend static FT to enable modeling of time dependent failures, introducing new dynamic gates and elements. DFT add a temporal notion to FT: system failures can depend on the order of component failures. DFT are a good attempt in dynamic reliability modeling, but they cannot adequately represent several of the dynamic behaviours previously listed. More specifically, using DFT it is hard (and in some cases not possible) to compose dependencies reflecting characteristics of complex and/or hierarchical systems, to define customizable redundancy schema or policy, to represent load sharing and probabilistic-common failure mode phenomena, and to adequately model reparability features. To overcome these problems or lacks in reliability/availability modeling, in Distefano & Xing 2006, Distefano & Puliafito 2006, Distefano et al. 2006 and Distefano 2005 we have defined a new reliability/availability modeling notation named *dynamic reliability block diagrams* (DRBD), by extending the RBD formalism.

In this paper we explain how to use the DRBD notation in system modeling and analysis. To do this we propose a methodology that, starting from the system structure, drives to the overall system availability analysis by evaluating the corresponding DRBD model. The methodology fixes the guidelines for modeling a system by DRBD and, subsequently, for analyzing the DRBD model thus obtained. Moreover, we try to specify the underlined methodology step by step, by parallely describing an example/case study of a multiprocessor distributed computing system as complement for the explanation, in order to clarify this latter. A comparison with the DFT approach is also provided showing how it is possible to map from the existing model types to the new model type. By this way, the capabilities, the flexibility, and the other features of the DRBD formalism in modeling and analysis of system reliability/availability could be adequately pointed out.

The reminder of the paper is organized as follows: section 2 presents the multiprocessor computing system taken as example to explain our methodology. In section 3 some background concepts of the utilized notations are provided. Section 4 schematically introduces the DRBD modeling approach and describes how to apply it to the motivating example; then, the analysis of the DRBD model thus obtained is reported in section 5, comparing our approach to the DFT one through the results. Lastly, section 6 provides some final considerations on the DRBD methodology.

2 MOTIVATING EXAMPLE: A MULTIPROCESSOR COMPUTING SYSTEM

In order to better describe the proposed methodology, we decided to adopt a step by step approach focused around the study of a system taken as case study.

It refers to a multiprocessor computing system drawn from literature (Malhotra & Trivedi 1995, Montani et al. 2006), from which we summarize the following description, accompanied by the scheme reported in Figure 1. It is composed by two computing module: CM_1 and CM_2. Each of them contains one processor (P_1 and P_2 respectively), one memory (M_1 and M_2) and two hard disks: a primary (D_{11} and D_{21}) and a backup disk (D_{12} and D_{22}). Initially, the primary disk is accessed by the computing modules, while the backup disk contains the copy of the information inside the primary disk, and is accessed only periodically for update operations. If the primary disk fails, it is replaced in its function by the backup disk. In terms of reliability the disks are identical, they are characterized by the same failure rate or reliability cumulative distribution function (cdf). But, when the primary disk is operational, the failure rate of the backup disk decreases as consequence of the management policy that decreases its workload condition.

The computing modules are connected by means of the bus N; moreover, P_1 and P_2 are energized by the power supply PS: the failure of PS forces P_1 and P_2 to fail.

M_3 is a spare memory replacing M_1 or M_2 in the case of failure. If M_1 and M_2 are operational, M_3 is just kept alive, but it is not accessed to load/store any data by the processors. When M_1 or M_2 fail, M_3 substitutes the failed unit.

In order to work properly the multiprocessors computing system of Figure 1 requires that at least one computing module (CM_1 or CM_2), the power supply PS and the bus N are operating correctly. Moreover a

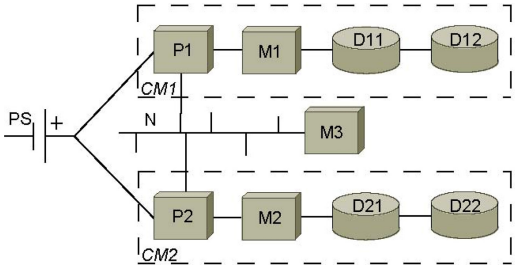

Figure 1. Schematic representation of the multiprocessor distributed computing system.

computing module (CM_1 and CM_2) is operational if the processor (P_1 and P_2 respectively), one between the local memory (M_1 and M_2) and the shared memory M_3 and one disk (D_{11} or D_{21} for CM_1 and D_{12} or D_{22} for CM_2) are not failed.

The example presented, which comes from Malhotra & Trivedi 1995 and Montani et al. 2006, where it is also analyzed, does not represent/implement the optimal system's management configuration available in terms of reliability. To improve the system reliability, a better solution could be to consider the two computing modules CM_1 and CM_2 as redundant subsystems. Another lack of the model is that no load sharing effects are considered between the computing modules: the fact that both CM_1 and CM_2 are operating could increase the reliability of each computing module, sharing the workload with the other one. If only a computing module works, it must process all the incoming requests, which were previously elaborated in cooperation with the other computing module. For this reason, the single *CM* could be more stressed and therefore its probability to fail could increase. These behaviours cannot be modeled by a DFT because they involve dependencies composition and/or modeling of uncovered aspects as the load sharing. Aspects that are instead adequately contemplated in the DRBD approach.

In this paper we describe how to model the multiprocessors computing system of Figure 1, using both DFT and DRBD in order to better explain the DRBD methodology and, at the same time, to compare the two approaches. In other words, the comparison between DRBD and DFT is subordinated and finalized to the DRBD methodology explanation, without specifying any mapping rules between DRBD and DFT, nor penetrating into the advanced DRBD modeling where behaviours not representable by DFT are investigated.

3 SYSTEM RELIABILITY MODELING NOTATIONS

To model and analyze the example proposed in the previous section, it is necessary to briefiy introduce the notations and the tools used to do this. In subsection 3.1 an overview of fault trees and dynamic fault trees is provided. Then, in subsection 3.2, we outline the reliability block diagrams notation to better introduce the dynamic reliability block diagrams, which are discussed in subsection 3.3. More details on the DRBD formalism can be found in Distefano & Xing 2006, Distefano & Puliafito 2006, Distefano et al. 2006, Distefano 2005.

3.1 *FT and DFT*

Fault trees provide a compact, graphical method to analyze system reliability. Static trees (Vesely et al. 1981) use Boolean gates to represent how component failures combine to produce system failure. Dynamic trees (Boyd 1991, Dugan & Doyle 1996, Dugan et al. 1992) add a temporal notion, by which the system failures can depend on the order of component failures. The static gates are therefore purely combinatorial logic gates applied to the input failure events. Component failure sequences are best captured using dynamic fault trees that extend static trees to enable modeling of time dependent failures. They can model dynamic replacement of failed components from pools of spares (CSP, WSP and HSP gates); failures that occur only if others occur in certain orders (PAND gates); dependencies that propagate the failure of one component to others (FDEP gates); and specification of constraints on failure orders that simplify analysis computations (SEQ gates).

3.2 *RBD*

The main virtue of a RBD is the ease in modeling and read. In a RBD, the logic diagram is arranged to indicate which combinations of component failures result in the failure of the system, or which combinations of properly working components keep the system operational. A block in RBD represents the working physical component, and the failure of this component is indicated by the removal of the corresponding block. If enough blocks are removed in an RBD to interrupt the connection between the input and output points, the system fails. In other words, if there is at least one path connecting input and output points, the system is still operating properly. Generally two main types of connection, series and parallel can be established between two or more components. The blocks in either series or parallel can be merged into a new block. Using such combinations, any parallel-series system can be eventually merged to one block and its reliability can be easily computed by repeatedly using the series and parallel structure equations.

3.3 *DRBD*

RBD ensure interesting features in reliability modeling such as simplicity, versatility and expressive power. Such interesting features are inherited in a DRBD model, which moreover allows taking into account the system dynamics. A system is considered time-variant, if its components states evolve when a sequence of events occur. It is possible to define reliability relationships (dependencies) among components by associating such relationships to events. DRBD implement this issue through two key points: characterizing each component by an own system dynamics and specifying the concept of dependency as the building block of dynamic reliability modeling.

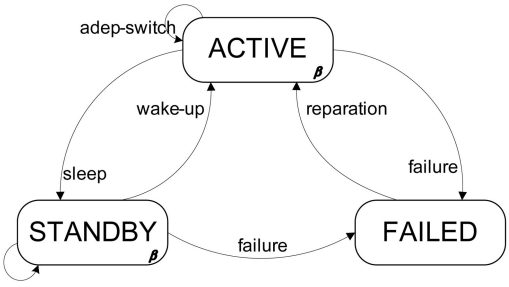

Figure 2. DRBD states-events machine.

Figure 3. DRBD order (a) and strong (b) dependencies representation.

In a DRBD model each component is characterized by a variable *state* identifying its operational condition at a given time. The evolution of a component's state (component's dynamics) is characterized by the *events* occurring to it. The states a generic DRBD component can assume are: *active* if the component works without any problem, *failed* if component is not operational, following up its failure, and *standby* if it is reliable but not available. Active components participate actively to perform the work or the task of the system, while standby components do not contribute to this, they do not interact with the other components as consequence of a *dependency* application. But, at the same time, a standby component is not failed, it just performs its internal activities.

An event represents the transition from a components' state to another one: the *failure* event from active or standby to the failed states, the *wake-up* switches from standby to active states, the *sleep* from active to standby states, the *reparation* from failed to active state, the *adep-switch/sdep-switch* represent the transitions between two active/standby states. These two latter events are related to the concurrency among dependencies. Figure 3 summarizes DRBD states and events.

The main enhancement introduced by DBRD is the capability to model dependencies among subsystems or components concerning their reliability behaviours. A dependency establishes a reliability relationship between two components or subsystems, a *driver* and a *target*. When a specified event, named *action* or *trigger*, occurs to the driver, the dependency condition is applied to the target. This condition is associated to a specific target event, named *reaction*. When a satisfied dependency condition becomes unsatisfied, the target component comes back to the fully active state.

A dependency could model two kinds of different relationships between a couple of driver-target components: the *order* (Figure 3 (a)) establishes the sequence order between trigger and reaction events, the *strong* (Figure 3 (b)) forces the target component to react when a trigger event occurs. A dependency is also characterized by the action/trigger and the reaction events. Four types of trigger and reaction events can be identified: wake-up (W), reparation (R), sleep (S) and failure (F). Combining action and reaction, 16 types of dependencies are identified for each relationship, 32 in total. Moreover, since the two relationships model two different behaviours, they can also be composed into more complex dependencies, identifying stronger conditions of dependence (Distefano 2005).

In the examples shown in Figure 3, A is the driver component and B the target. The dependency action or trigger event is indicated by a letter (W, R, S or F as above), and the reaction by another letter (from the same set), separated from the action by a slash. The total string is placed near the circle. In case of order dependencies an arrow is placed inside the circle (Figure 3 (a)), while a number characterizes strong dependencies (Figure 3 (b)): it indicates the *dependency rate* β. This latter characterizes strong dependencies with wake-up (W) and/or sleep (S) reaction, weighting, in terms of reliability, the dependence of target component from driver.

The concept of dependence is exploited in DRBD as the basis to represent any dynamic reliability behaviours, as those listed in section 1. For example, redundancy can be easily represented by exploiting and combining dependencies among units. Other possible applications of dependencies in dynamic system reliability modeling could be load sharing, common cause failure, reparation, and so on. Further details of these interesting capabilities of DRBD are out of the scope of the present work and can be found in Distefano & Xing 2006, Distefano & Puliafito 2006, Distefano et al. 2006, and Distefano 2005.

4 THE MODELING

This section goes inside the system reliability modeling problem. In subsection 4.1 the DRBD modeling approach is schematized by a five steps algorithm, then, the modeling of the multiprocessor computing system introduced in section 2 is described by mean of DFT (in subsection 4.2) and DRBD (subsection 4.3) in order to compare the two modeling approaches.

4.1 *DRBD modeling*

The DRBD modeling approach is implemented by a sequence of actions/steps summarized in the following algorithm. It is based and/or derived from the

"immediate cause" concept methodology proposed in Vesely et al. 1981 for FT, adapted to the DRBD modeling by also considering the system's dependencies/dynamics. According to the "immediate cause" concept algorithm, the modeler firstly identifies the system top event, then he determines the immediate, necessary and sufficient causes for its occurrence. Such events are then treated as sub-top events, to which apply recursively the "immediate cause" concept, determining, for each of them, the necessary and sufficient causes of failure, until the basic events are reached.

The deriving DRBD algorithm can be schematized in five steps:

1 *System specification*: the overall system is defined from a reliability viewpoint, also individuating the system reliability domain (the corresponding of the DFT top event).
2 *Subsystems identification*: the immediate, necessary and sufficient causes for the system failure are individuated, identifying system faults and corresponding subsystems. Each subsystem is composed by all the components and the subsystems driving to the related fault.
3 *Structural linking*: each subsystem identified in the previous step is linked to the others according to the structural reliability relationships. If the subsystem reliability is necessary to the system reliability, or the subsystem fault drives to the system failure, the subsystems are series connected; otherwise, if this is sufficient, the subsystems implement a parallel structures.
4 *Dynamic linking*: the subsystems identified in step 2 are connected to the other subsystems from which depend by exploiting DRBD dependencies. The impact and the priority (in case of conflicts among incoming dependencies) of dependencies must be adequately weighted/evaluated through the β and p parameters respectively (see subsection 3.3).
5 *Reiteration*: the algorithm iterates from step 2, by considering a subsystem by time until the components detail is reached.

The DRBD modeling therefore consists in an iterative process that, starting from the system reliability specifics, lead to the DRBD model through several phases that progressively refine the system and its subsystems. In the first phase the overall system is considered; by proceeding with the underlined algorithm, the abstraction level is lowered until the components details are specified.

4.2 *The DFT model*

The DFT modeling the multiprocessor computing system is depicted in Figure 4. In it the time to failure of any component of the system is represented as a random variable, associated to the corresponding block in the scheme of Figure 1. The DFT model is composed by a FDEP and four WSPs. The FDEP gate models the functional dependency among the power supply (PS) and the two processors P_1 and P_2. WSP_1 and WSP_4 represent the hard disks management policy: the primary disks D_{11} and D_{21} drive WSP_1 and WSP_4 gates respectively in the control of the corresponding backup disks D_{12} and D_{22}. In other words, the backup disks D_{12} and D_{22} are considered as spare units of the primary disks D_{11} and D_{21} respectively. From the probabilistic/analytical point of view, this choice well describes the reliability behaviour of the components, but, from a semantic viewpoint, it does not adequately represent the real condition: the backup disks do not assume a standby configuration because they communicate with the primary disks, making active operations. A more realistic modeling should therefore take into account this fact.

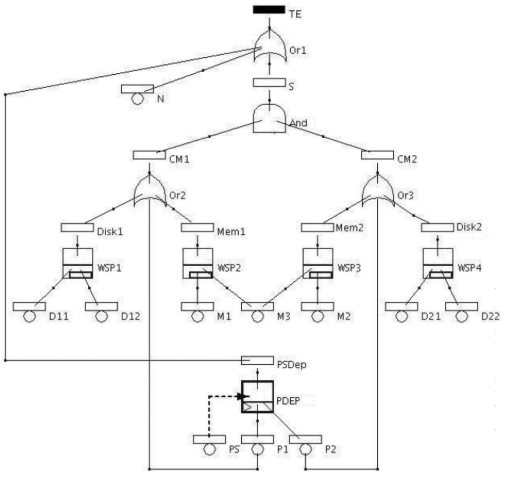

Figure 4. DFT model of the multiprocessor distributed computing system.

The *partly-loaded standby redundancy policy* applied to the memory units M_1, M_2 and M_3, is represented by the WSP_2 and WSP_3 gates: if M_1 and M_2 fail, M_3 is activated. The other DFT gates are static: the internal events $DISK_1$ and $DISK_2$ represent the failure of the storage blocks related to the computing modules CM_1 and CM_2 respectively, analogously MEM_1 and MEM_2 represent the computing modules' memory block failure. The failure of the processor (P_1 and P_2) or of the memory block (MEM_1 and MEM_2) or of the disk block ($DISK_1$ and $DISK_2$) drives to the failure of the corresponding computing module (CM_1 and CM_2 internal events). Finally, if both the computing modules fail, or the power supply PS goes down, or the bus N fails, the overall system fault occurs, represented in the DFT as the top event TE.

4.3 *The DRBD model*

By applying the algorithm described in subsection 2.1 to the example under consideration, the DRBD model reported in Figure 5 is obtained. This process starts by identifying the system as a multi-processors computing system, that is analyzed in order to obtain the overall system reliability. In the first iteration of the algorithm, three possible faults could be identified for the system (step 2): fault in energizing, fault in computing and fault in interconnection. The first regards the power supply component *PS*, the second the computing subsystem *CS* and the third is associated to the bus N. If all the three subsystems are operating the system is reliable, thus, implementing step 3, they are series connected. Since the three subsystems are independent (the step 4 is bypassed), the algorithm reiterates (step 5), taking into consideration the computing subsystem because the other subsystems are components.

CS could be affected by two faults, corresponding to the two computing modules CM_1 and CM_2 failures respectively. The CM_1 and CM_2 subsystems are redundant and therefore parallel connected; moreover they are not dependent, thus the algorithm iterates to the computing modules investigation. A computing module has three possible faults: processing, memory and disk's faults. All of them represent computing module's failure modes, thus the three corresponding subsystems, the processor (P_1 for CM_1 or P_2 for CM_2) the memories (M_1–M_3 or M_2 – –M_3 respectively) and the disks (D_{11}–D_{21} or D_{12}–D_{22}), are series connected. They are also not dependent from any other subsystems.

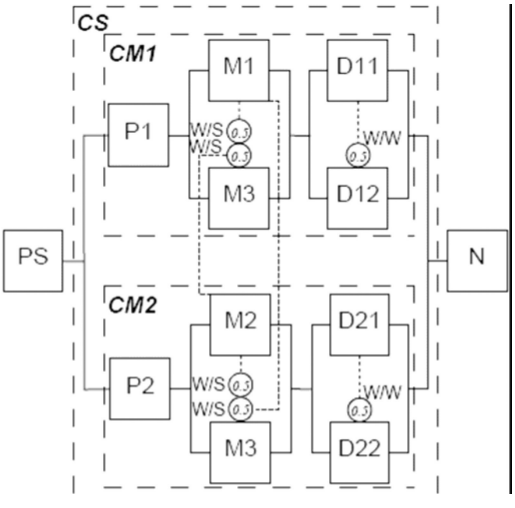

Figure 5. DRBD model of the multiprocessor distributed computing system.

Reiterating the algorithm, the memory subsystems are considered. Two are the possible faults affecting these subsystems that are related to the M_1 and M_3 components in case of the subsystem CM_1 or to the M_2 and M_3 components considering CM_2. It is not necessary that both memories are operating in order that a memory subsystem is operating, thus they are connected in parallel. Moreover the M_3 component is managed as a spare unit, thus a strong dependency is represented from M_1 (M_2) to M_3.

More specifically, wake-up/standby strong dependencies are exploited to model the redundancy policy managing the memories. These represent the disabling conditions applied by M_1 and M_2, when they are operational, on M_3, keeping this latter in standby. The overall dependency must be applied to M_3 if and only if both M_1 and M_2 are at the same time operational; when one of these fails, M_3 must switch to the fully active state. To realize this condition the two wake-up/standby strong dependencies, from M_1 to M_3 and from M_2 to M_3, are *series composed* (Distefano 2005): when both are simultaneously satisfied the component M_3 is placed in standby, otherwise M_3 is active. This composed dependency is duplicated in the DRBD model for the sake of clarity, identifying and separating the two computing modules CM_1 and CM_2.

The disks subsystems, as the memories subsystems, have two main faults associated to the respective disks D_{11} and D_{12} for CM_1 and D_{21} and D_{22} for CM_2. The two disks are redundant, therefore they are parallel connected; moreover they are also dependent for the standby redundancy policy applied. The disks management policy is represented in DRBD by a wake-up/wake-up strong dependency: when the primary disks D_{11} and/or D_{21} are operational, the backup disks D_{12} and/or D_{22} respectively are partial active, maintaining the backup. The level of activity of the dependent components is numerically translated into the DRBD by the dependency rate β (see subsection 3.3), that, in this case, can assume a value in the range between 0 and 1 representing decreasing failure rate conditions. When a primary disk fails, the corresponding backup disk that substitutes it becomes fully active and fully energized since the dependency condition becomes unsatisfied.

This completes the DRBD modeling process, done by following the guidelines specified in subsection 4.1.

In the DRBD model reported in Figure 5, each device of the system is identified by the corresponding component name and it is also characterized by the corresponding reliability cdf. Since the power supply *PS* energizes the two processors P_1 and P_2, we interpret this behaviour, and consequently represent it in the DRBD model, as a simple series between each processor and the *PS*, as detailed above. As in the case of the warm spare disks discussed in the previous subsection, this is just a semantic imperfection of

the DFT model, in the fact that the failure of *PS* generally does not imply the processors failure; in other words this behaviour does not implement a common failure mode condition. Anyway, exploiting the same principle applied for DFT modeling, this could be a good solution to map a FDEP gate into the DRBD domain for reliability evaluations, in the case that no reparability features must be modeled.

5 THE ANALYSIS

The purposes of this section are to describe how a DRBD can be analyzed, also confronting the DRBD approach to the DFT one in order to demonstrate the effectiveness of the former. Firstly, in subsection 5.1, an overview of the possible DRBD solution techniques is introduced, specifying the solution algorithm then applied, as reported in subsection 5.2, in the analysis.

5.1 *DRBD Analysis*

The DRBD formalism is a powerful notation to study system reliability deriving from RBD. If the independence assumption stands, i.e. each component of the DRBD model is stochastically independent from the others, the DRBD model can be considered as a RBD and therefore analyzed by applying the combinatorial structures equations (Rausand & Høyland 2003), obtaining the total reliability function analytically. Unfortunately the combinatorial/analytic method cannot be applied or extended to DRBD models with dependence among components. In these cases it is necessary to map the DRBD model onto an analyzable domain, implementing a two step methodology where the DRBD model is an intermediate model interposed between the modeler and the analysis domain, as also done in DFT. Therefore, referring to the DFT case for analogy and inspiring on literature (Manian et al. 1998, Malhotra & Trivedi 1993, Trivedi et al. 1996, Coppit et al. 2000), a wide range of possibilities are available for analyzing a *"dynamic"* DRBD model by translating it into an analysis domain. As introduced in section 1, the choice is between *analytic* or *simulation* methods. For what regards the analytic methods there are many possible alternatives, according to the nature of the reliability cdfs or the overall complexity. The most widely used is the continuous time Markov chain (CTCM), but it is limited in the reliability cdfs tractable. A more general approach should be based on Petri nets (PN), queuing networks, Bayesian networks (BN) or similar analytic formalisms. For example the approaches described in Volovoi 2004 and Malhotra & Trivedi 1995 are applications of the PN formalism in DFT analysis. Other interesting techniques to analyze a DFT exploit stochastic reward nets (Zuppala et al. 1994, Malhotra & Trivedi 1995), Markov reward models (Veeraraghavan & Trivedi 1994), Bayesian networks (Zhou et al. 2006), stochastic well-formed nets (Bobbio et al. 2003) and queuing networks (Bolch et al. 2006). Some of these methodologies remove the restrictions related to the CTMC methods.

As regards DRBD, we have developed a solution based on CTMC, with the restrictions mentioned above. On the other hand Markov models and DRBD implement two different approaches: the former identifies all the states and the related transitions characterizing the system, while the latter describes the system from an higher level of abstraction, the component view. Thus, the two models could be complementary: the DRBD can be exploited for modeling the system while the Markov model derived from the DRBD can be used in the analysis phase. Another difference between the two approaches is represented by the reliability distribution functions considered: while Markov assumption is limitative, DRBD try to solve any kind of problem, also considering aging effects and generally distributed reliability cdfs. In order to implement this issue and/or to overcome the Markov model restrictions we are investigating a solution based on non-Markovian stochastic PN (NMSPN) (Bobbio et al. 2000), applied in Distefano et al. 2006 and here to analyze the motivating examples, but in the actual version also restricted to the constant failure rate assumption. For more details refer to Distefano 2005.

An attractive alternative to analytic approaches is the simulation (Wang et al. 2004, Figiel & Sule 1990), because it allows the modeling of any reliability distribution without any particular restrictions. One of the drawback of simulation is the long elaboration time needed to achieve accuracy in the solution. However, using variance reductions techniques the elaboration time could be dramatically reduced. The great part of the simulation methodologies applied to the DFT analysis (Manian et al. 1998, Gedam & Beaudet 2000), exploit the *Monte Carlo* technique. Since the Monte Carlo simulation approach is very flexible, we retain it could be successfully applicable to the DRBD analysis and we are actually investigating in this way.

However the optimal solution is to combine different techniques (Veeraraghavan & Trivedi 1994, Manian et al. 1998) by adequately decomposing the DRBD model into independent parts or subsystems. The parts involving dependencies are identified as *dynamic subsystems*. The others are identified as *static subsystems*. The static subsystems are analyzed using the combinatorial/analytic equations, while the dynamic subsystems can be analyzed referring to one of the methods introduced above, according to the nature of the reliability distributions and the complexity of the considered part. After a separated elaboration of each subsystem, the results must be merged by

Table 1. Parameters related to the multiprocessors computing system example.

Component	λ	α	β
N	2		
P_1, P_2	500		
PS	6000		
D_{11}, D_{21}	80000	0.5	0.5
D_{12}, D_{22}	80000	0.5	0.5
M_1, M_2	30		
M_3	30	0.5	0.5

applying the structure equations to the subsystems. This algorithm can be summarized in three steps:

1 *Split the DRBD*: in as much static and dynamic subsystems as possible. Each subsystem must be independent from the others.
2 *Analyze the subsystems*: the static ones are analyzed by applying the RBD structure equations, while the dynamic parts are mapped onto an analytic/simulation method, evaluating case by case, according to the nature of the reliability functions contained in the subsystem, the complexity of the subsystem and the expected accuracy.
3 *Rejoin the results*: coming from the different elaborations by applying the RBD structure equations.

5.2 *Results*

The example described in section 2 and modeled in section 4, has been studied in depth by analyzing the overall system reliability cdf trend in time, knowing the corresponding failure rates. All the components have been modeled by a constant failure rate λ characterizing exponential reliability cdfs or *memoryless* systems.

The values contained in Table 1, drawn from literature (Malhotra & Trivedi 1995, Montani et al. 2006) as the motivating example, report the parameters used to analyze this latter. λ, as introduced above, is the failure rate, α indicates the dormancy factor and β the dependency rate of the dependency between the corresponding components, where $\beta = 1 - \alpha$. The λ values are expressed in *failures in time* (FITs), i.e. number of faults per billion device hours (1 FIT = 1 fault/10^9 hours).

By applying the underlined algorithm to the multiprocessors computing systems DRBD reported in Figure 5, this can be subdivided in three subsystems: the first, static, is composed by the series among the power supply *PS* and the bus *N*, series connected with the parallel between the two computing modules $CM1$ and $CM2$ that are the other two subsystems. Since these latter are identical, it is possible to study only one

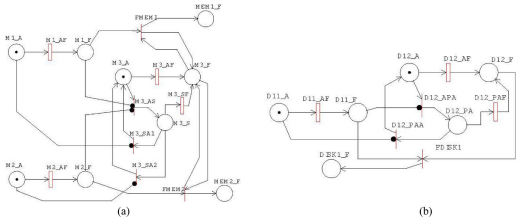

Figure 6. GSPNs modeling the memory (a) and the disk (b) subsystems.

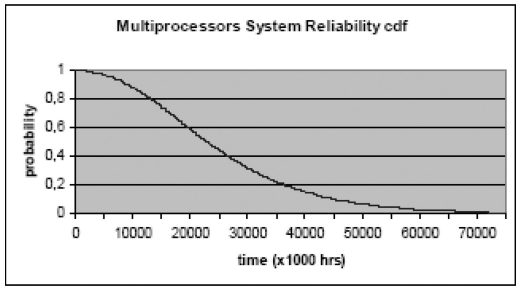

Figure 7. Trend of the system reliability cdf.

computing module subsystem and then apply the parallel structure equation to obtain the reliability of the parallel between the two computing modules. A computing module is further subdivided onto the series of three blocks: the processor, the memory and the disk. The memory and the disk blocks are dynamic parts. To study these dynamic parts the generalized SPNs (GSPNs) reported in Figure 6 are exploited. Analyzing the two GSPNs through the WebSPN tool (Scarpa et al. 2003) and putting all together by applying the RBD structures equations, the results shown in Figure 7 are obtained.

Figure 7 reports the overall multiprocessors computing system reliability cdf.

By the same way, the DFT model depicted in Figure 4 corresponding to the motivating example discussed, has been analyzed in (Montani et al. 2006) by exploiting three different tool: DBNet (Montani et al. 2006), DRPFTproc (Bobbio et al. 2003) and Galileo (Sullivan et al. 1999). The first analyzes the DFT by translating it into a dynamic Bayesian network (DBN) and therefore by solving the DBN. DRPFTproc is based on modularization and conversion to stochastic well-formed nets (SWN) of the dynamic gates, tracing back the problem to a SWN solution. Galileo approaches the problem by firstly modularizing it, then solving the obtained modules by exploiting binary decision diagrams and CTMCs.

The results obtained by such analysis are summarized in Table 2, where they are compared each other

Table 2. Unreliability results obtained by the multiprocessors computing system example analysis.

Time	DBNet	DRPFTproc	Galileo	DRBD
1000	0.006009	0.006009	0.006009	0.006009
2000	0.012245	0.012245	0.012245	0.012245
3000	0.019182	0.019183	0.019183	0.019183
4000	0.027352	0.027355	0.027355	0.027354
5000	0.037238	0.037241	0.037241	0.037240

also referring to the DRBD approach. Table 2 summarizes some system unreliability probabilities calculated in specific time instants. The time is expressed in hours. These results demonstrate and validate the effectiveness of the DRBD approach, providing consistent values for all the tests.

6 CONCLUSIONS

In this paper the effectiveness of the DRBD methodology in representing and analysis dynamic reliability/availability aspects, is demonstrated by exploiting a multiprocessors computing system example drawn from literature. Basing on this latter the DRBD methodology is explained step by step, also establishing a deep comparison with the DFT approach.

The problem to analyze a DRBD model is faced in the paper, describing an interesting algorithm that combines different solutions. This algorithm is applied to the multiprocessors system analysis, in order to obtain the overall system reliability. These results compared to the one obtained by the DFT analysis, allow to identify the DRBD as a valid alternative in dynamic reliability/availability analysis scenario, also considering its potentiality in flexibility, modeling power and the other capabilities.

REFERENCES

Reliasoft Corporation 2003. *System Analysis Reference: Reliability Availability and Optimization*. Reliasoft Publishing.

Gedam, S. G. & Beaudet, S. 2000. Monte Carlo simulation using Excel® spreadsheet for predicting reliability of a complex system. *Reliability and Maintainability Symposium*: pp. 188–193.

Kamal, H. & Ayyub, B. M. 1999. Reliability assessment of structural systems using discrete-event simulation. *Proceedings of 13th ASCE Engineering Mechanics Division Specialty Conference*. ESRA.

Trivedi, K. S. 2001. *Probability and Statistics with Reliability, Queuing and Computer Science Applications*. 2nd ed. Chichester, UK. John Wiley and Sons Ltd.

Veeraraghavan, M. & Trivedi, K. S. 1994. A combinatorial algorithm for performance and reliability analysis using multistate models. *IEEE Trans. Computers*. Vol. 43, no. 2: pp. 229–234.

Bouissou, M. & Bonc, J. L. 2003. A new formalism that combines advantages of fault trees and Markov models: Boolean logic driven Markov processes. *Reliability Engineering and System Safety*. Vol. 82, no. 2: pp. 149–163.

Malhotra, M. & Trivedi, K. S. 1995. Dependability modeling using Petri-nets. *IEEE Transaction on Reliability*. Vol. 44, no. 3: pp. 428–440.

Bobbio, A., Franceschinis, G., Gaeta, R., Portinaie, L. 2003. "Parametric fault tree for the dependability analysis of redundant systems and its high-level Petri net semantics. *IEEE Trans. Software Engineering*. Vol. 29, no. 3: 270–287.

Zuppala, J., Ciardo, G., Trivedi, K. S. 1994. Stochastic reward nets for reliability prediction. *Communications in Reliability, Maintainability and Serviceability*. Vol. 1, no. 2: 9–20.

Zhou, Z., Jin, G., Dong, D., Zhou, J. 2006. Reliability analysis of multistate systems based on Bayesian networks. *Proceedings of the 13th Annual IEEE International Symposium and Workshop on Engineering of Computer Based Systems (ECBS'06)*. Washington, DC, USA: IEEE Computer Society: pp. 344–352.

Montani, S., Portinaie, L., Bobbio, A., Reiteri, D. C. 2006. Automatically translating dynamic fault trees into dynamic bayesian networks by means of a software tool. *Proceedings of the The First International Conference on Availability, Reliability and Security, ARES 2006*. IEEE Computer Society: pp. 804–809.

Rausand, M. & Høyland, A. 2003. *System Reliability Theory: Models, Statistical Methods, and Applications*. 3rd ed. Wiley-IEEE.

Vesely, W.E., Goldberg, F. F., Roberts, N. H., Haasl, D. F. 1981. *Fault Tree Handbook*. U. S. Nuclear Regulatory Commission, NUREG-0492, Washington DC.

Sahner, R., Trivedi, K. S., Puliafito, A. 1996. *Performance and Reliability Analysis of Computer Systems: An Example-based Approach Using the SHARPE Software Package*. Kluwer Academic Publisher.

Boyd, M. A. 1991. *Dynamic fault tree models: Techniques for analysis of advanced fault tolerant computer systems*. Ph.D. thesis, Duke University, Department of Computer Science.

Dugan, J. B. & Doyle, S. A. 1996. New results in fault-tree analysis. *Reliability and Maintainability Symposium tutorial Notes*. pp. 568–573.

Distefano, S. & Xing, L. 2006. A new modeling approach: Dynamic Reliability Block Diagrams. *Proceedings of the 52nd Annual Reliability and Maintainability Symposium (RAMS06)*.

Distefano, S. & Puliafito A. 2006. System modeling with dynamic reliability block diagrams. *Proceedings of the Safety and Reliability Conference (ESREL06)*. ESRA.

Distefano, S., Scarpa, M., Puliafito, A. 2006. Modeling distributed computing system reliability with DRBD. *Proceedings of the 25th IEEE Symposium on Reliable Distributed Systems (SRDS'06)*. Washington, DC, USA. IEEE Computer Society: pp. 106–118.

Distefano, S. 2005. *System dependability and performances: Techniques, methodologies and tools*. Ph.D. thesis. University of Messina.

Dugan, J. B., Bavuso, S., Boyd M. 1992. Dynamic fault tree models for fault tolerant computer systems. *IEEE Transactions on Reliability*. Vol. 41, no. 3: pp. 363–377.

Manian, R., Dugan, J. B., Coppit, D., Sullivan, K. J. 1998. Combining various solution techniques for dynamic fault tree analysis of computer systems. *Proceedings of IEEE International High-Assurance Systems Engineering Symposium*.

Malhotra, M. & Trivedi, K. S. 1993. Reliability and performability techniques and tools: a survey. *MMB*: pp. 27–48.

Trivedi, K. S., Hunter, S., Garg, S., Fricks, R. 1996. Reliability analysis techniques explored through a communication network example. *International Workshop on Computer-Aided Design, Test, and Evaluation for Dependability*.

Coppit, D., Sullivan, K. J., Dugan, J. B. 2000. Formal semantics for computational engineering: A case study on dynamic fault trees. *ISSRE*: pp. 270–282.

Volovoi, V. V. 2004 "Modeling of system reliability using Petri nets with aging tokens," Reliability Engineering and System Safety, Vol. 84, no. 2: pp. 149–161.

Bolch, G., Greiner, S. De Meer, H., Trivedi, K. S. 2006, *Queuing Networks and Markov Chains: Modeling and Performance Evaluation with Computer Science Applications*. 2nd ed. Wiley-Interscience.

Bobbio, A., Puliafito, A., Telek, M. 2000. "A modeling framework to implement preemption policies in non-Markovian SPNs. *IEEE Transaction on Software Engineering*. Vol. 26, no. 1: pp. 36–54.

Wang, W., Loman, J. M., Arno, R. G., Vassiliou, P., Furlong, E. R., Ogden, D. 2004. Reliability block diagram simulation techniques applied to the IEEE std. 493 standard network. *IEEE Transaction on Industry Applications*. Vol. 40, no. 3: pp. 887–895.

Figiel, K. D. & Sule, D. R. 1990. A generalized reliability block diagram (RBD) simulation. *Simulation Conference*: pp. 551–556.

Scarpa, M., Puliafito, A., Distefano, S. 2003. A parallel approach for the solution of non Markovian Petri Nets. *10th European PVM/MPI Users' Group Conference (EuroPVM/MPI03)*. Springer Verlag LNCS 2840: pp. 196–203.

Sullivan, K. J., Dugan, J. B., Coppit, D. 1999. The Galileo fault tree analysis tool. *Proceedings of the 29th Annual International Symposium on Fault-Tolerant Computing*. IEEE: pp. 232–5.